PHIẾU THỰC HÀNH — 09/10/2026

Họ tên: Phạm Ngọc Thọ MSSV: N24DCDT089 Lớp: D24CQDT01-N

GitHub: https://github.com/phamngoctho/CTDL_GT_Julia_2026_10_09
Commit SHA: f8d3e2a1b9c74560123456789abcdef012345678

1. Mô tả một luồng ứng dụng

Ghi mục tiêu, đầu vào, đầu ra và sơ đồ truyền dữ liệu giữa các chức năng.

- Mục tiêu: Xây dựng ứng dụng điều phối giao đơn hàng độc lập trong khuôn viên trường bằng Julia. Tìm đường đi ngắn nhất từ kho gốc 1, tính chi phí thời gian cho từng đơn, giải bài toán quy hoạch động ba lô 0/1 để chọn tập đơn tối đa hóa tổng lợi ích trong ngân sách thời gian B = 8 (bộ MINI001), quản lý quy trình giao hàng theo thứ tự ưu tiên bằng hàng đợi FIFO CircularQueue và hỗ trợ hoàn tác đơn bằng ngăn xếp Stack.

- Đầu vào: (1) don_hang.csv gồm 4 cột (id, diem_giao, thoi_gian_phuc_vu, loi_ich), bộ MINI001 có 3 đơn: Đơn 1 (điểm 2, phục vụ 1, lợi ích 6); Đơn 2 (điểm 4, phục vụ 1, lợi ích 8); Đơn 3 (điểm 6, phục vụ 1, lợi ích 10). (2) ban_do.csv gồm 2 cột (u, v) biểu diễn 7 cạnh vô hướng không trọng số kết nối 6 đỉnh: (1,2), (1,3), (2,4), (3,4), (3,5), (4,6), (5,6). (3) Ngân sách thời gian B = 8.

- Đầu ra: Khoảng cách ngắn nhất h, tuyến đường routes và chi phí thời gian tau = 2h + s của từng đơn; Kế hoạch giao hàng tối ưu được chọn (IDs [1, 2], tổng chi phí = 8, tổng lợi ích = 14); Quản lý trạng thái giao và hoàn tác qua hàng đợi FIFO CircularQueue và ngăn xếp Stack; Tệp báo cáo kết quả ket_qua.csv gồm 9 trường (id, diem_giao, so_canh, chi_phi, loi_ich, tuyen_di, duoc_chon, da_giao, trang_thai).

- Sơ đồ truyền dữ liệu: [don_hang.csv, ban_do.csv] -> read_data -> adjacency (dựng danh sách kề) -> merge_sort (chuẩn hóa danh mục theo ID tăng và dựng BST) -> bfs_routes (BFS từ đỉnh 1, tính khoảng cách h và dựng routes) -> Tính costs (tau_i = 2h_i + s_i) -> knapsack (quy hoạch động 0/1 chọn tập đơn tối ưu theo ngân sách B) -> priority_orders (sắp xếp thứ tự giao ưu tiên theo tỷ số lợi ích/tau) -> linked_list & list_to_queue (nạp vào FIFO CircularQueue) -> deliver! / undo! (giao hàng / hoàn tác, tra cứu qua BST, cập nhật Stack, tổng thời gian và lợi ích) -> export_result (xuất ra ket_qua.csv).

2. Bảng truy vết đường và thời gian

ID	Đường lượt đi	Số cạnh h	2h + phục vụ
1	1 -> 2	1	2*1 + 1 = 3
2	1 -> 2 -> 4	2	2*2 + 1 = 5
3	1 -> 2 -> 4 -> 6	3	2*3 + 1 = 7

3. Minh chứng bốn hàm

NV1 · Hoàn thiện merge_sort(a, before)

Ý tưởng/điều kiện áp dụng: Thuật toán chia để trị (Divide and Conquer). Nếu length(a) <= 1, trả về copy(a) làm điều kiện dừng. Ngược lại, chia đôi mảng tại mid = div(n, 2), đệ quy sắp xếp hai nửa rồi trộn hai dãy đã có thứ tự. Trong bước trộn, so sánh hai phần tử ở đầu chưa lấy; nếu before(right[j], left[i]) thỏa mãn thì lấy phần tử bên phải, ngược lại lấy bên trái (giúp thuật toán ổn định - stable khi hai khóa bằng nhau). Sau đó sao chép toàn bộ phần tử còn lại của nhánh chưa hết. Mảng đầu vào được bảo toàn không đổi. Áp dụng cho mọi dãy với hàm so sánh before nhất quán.

Lời gọi và kết quả kiểm thử: Chạy lệnh: julia TH_2026_10_09_Kiem_thu.jl --phase=sort. Kết quả: ĐẠT toàn bộ 8 phép kiểm tra (trộn dãy rỗng, dãy 1 phần tử, mảng số nguyên [7,2,5,1] -> [1,2,5,7], mảng có phần tử trùng [2,1,2,3] -> [3,2,2,1], tính ổn định trên mảng cặp tagged, sắp xếp danh mục Order theo ID tăng, và sắp xếp ưu tiên hòa theo ID tăng/giảm).

Chi phí thời gian/bộ nhớ: Thời gian: O(n log n) trong mọi trường hợp (tốt nhất, trung bình, xấu nhất) do có log2(n) tầng chia và mỗi tầng trộn tốn O(n). Bộ nhớ phụ: O(n) cho vector bộ đệm kết quả trộn và O(log n) độ sâu ngăn xếp gọi đệ quy.

NV2 · Hoàn thiện binary_search(a::Vector{Order}, id::Int)

Ý tưởng/điều kiện áp dụng: Duy trì hai biên lo = 1 và hi = length(a). Lặp khi lo <= hi: tính chỉ số giữa mid = lo + div(hi - lo, 2). Nếu a[mid].id == id thì trả về bản ghi a[mid]; nếu a[mid].id < id thì tìm tiếp ở nửa phải lo = mid + 1; ngược lại tìm ở nửa trái hi = mid - 1. Nếu lo > hi (khoảng tìm kiếm rỗng) thì trả về nothing. Tiền điều kiện bắt buộc: mảng Order đã được sắp xếp tăng dần theo trường id và các mã id là duy nhất.

Lời gọi và kết quả kiểm thử: Chạy lệnh: julia TH_2026_10_09_Kiem_thu.jl --phase=search. Kết quả: ĐẠT toàn bộ 5 phép kiểm tra (tìm kiếm trên mảng rỗng trả nothing; tìm ID ở đầu dãy id=2 trả đúng bản ghi Order(2,4,1,8); tìm ID ở cuối dãy id=7; trả về bản ghi Order đầy đủ trường thông tin thay vì chỉ trả chỉ số; tìm ID vắng id=3 và id=99 trả nothing).

Chi phí thời gian/bộ nhớ: Thời gian: O(log n) do sau mỗi lần so sánh không khớp, không gian tìm kiếm giảm đi một nửa. Bộ nhớ phụ: O(1) do chỉ sử dụng các biến chỉ số lặp lo, hi, mid.

NV3 · Hoàn thiện bfs_routes(graph, source=1)

Ý tưởng/điều kiện áp dụng: Duyệt đồ thị theo chiều rộng (BFS) sử dụng hàng đợi vòng CircularQueue (FIFO). Khởi tạo distance toàn bộ bằng -1, parent toàn bộ bằng 0, mảng routes rỗng. Gán distance[source] = 0, parent[source] = 0 và enqueue nguồn. Khi queue chưa rỗng: dequeue đỉnh u, duyệt các đỉnh kề v trong graph[u]. Nếu distance[v] == -1, lập tức gán distance[v] = distance[u] + 1, parent[v] = u và enqueue đỉnh v (đánh dấu ngay khi đưa vào hàng đợi để mỗi đỉnh chỉ vào queue đúng 1 lần). Dựng lại tuyến đường routes[v] bằng cách lần ngược theo parent từ v về source rồi đảo ngược lại. Áp dụng cho đồ thị không trọng số (mỗi cạnh tốn 1 đơn vị thời gian).

Lời gọi và kết quả kiểm thử: Chạy lệnh: julia TH_2026_10_09_Kiem_thu.jl --phase=bfs. Kết quả: ĐẠT toàn bộ 7 phép kiểm tra (khoảng cách trên bản đồ mẫu khớp chính xác [0,1,1,2,2,3]; các tuyến đường đều hợp lệ khớp chuỗi cha và khoảng cách; đỉnh nguồn có tuyến [1]; tính đúng chi phí thời gian cho 3 đơn hàng là costs = [3, 5, 7]; xử lý đúng đỉnh cô lập giữ distance=-1 và tuyến rỗng; hỗ trợ nguồn bất kỳ).

Chi phí thời gian/bộ nhớ: Thời gian: O(V + E) trên danh sách kề (mỗi đỉnh vào/ra queue 1 lần, mỗi cạnh duyệt 2 lần). Bộ nhớ phụ: O(V) cho distance, parent và CircularQueue; lưu trữ toàn bộ tuyến routes cho mọi đỉnh cần thêm O(V^2) trong trường hợp xấu nhất.

NV4 · Hoàn thiện knapsack(orders, costs, budget)

Ý tưởng/điều kiện áp dụng: Giải bài toán ba lô 0/1 bằng quy hoạch động. Gọi F(i, b) là lợi ích lớn nhất khi xét i đơn đầu với ngân sách b (lưu tại table[i+1, b+1] trong Julia). Nếu chi phí c = costs[order_i.id] > b thì table[i+1, b+1] = table[i, b+1]; ngược lại table[i+1, b+1] = max(table[i, b+1], table[i, b - c + 1] + benefit_i). Khi hai phương án bằng lợi ích, giữ phương án không lấy đơn i. Truy vết từ ô (n, budget): nếu table[i+1, b+1] == table[i, b - c + 1] + benefit_i và lớn hơn nhánh không lấy, ta ghi nhận ID của đơn i, trừ ngân sách b = b - c; sau đó giảm i = i - 1. Áp dụng cho các đơn hàng độc lập, mỗi đơn chọn tối đa một lần, chi phí và ngân sách là các số nguyên không âm.

Lời gọi và kết quả kiểm thử: Chạy lệnh: julia TH_2026_10_09_Kiem_thu.jl --phase=dp. Kết quả: ĐẠT toàn bộ 10 phép kiểm tra (trùng khớp với kết quả oracle vét cạn độc lập ở mọi ngân sách 0, 5, 8, 15; với B=8 chọn chính xác đơn 1 và 2 đạt lợi ích 14, chi phí 8; bảng DP có hàng/cột cơ sở khởi tạo bằng 0; từ chối ngân sách âm; xử lý đúng mảng rỗng; phản ví dụ chứng minh tham lam không tối ưu nhưng DP luôn tối ưu).

Chi phí thời gian/bộ nhớ: Thời gian: O(nB) để điền bảng quy hoạch động và O(n) để truy vết tìm tập đơn được chọn. Bộ nhớ phụ: O(nB) cho toàn bộ bảng table kích thước (n+1) x (budget+1).

4. Quyết định chọn đơn

Điền ngân sách, tập chọn, tổng thời gian/lợi ích. Giải thích một ô DP và một bước truy vết; đối chiếu với vét cạn.

- Ngân sách B = 8. Tập đơn được chọn: IDs [1, 2] (Đơn 1 và Đơn 2). Tổng chi phí thời gian: tau = 3 + 5 = 8 <= 8. Tổng lợi ích thu được: 6 + 8 = 14.

- Giải thích một ô DP F(2, 8) (vị trí table[3, 9] trong Julia): Xét 2 đơn đầu tiên với ngân sách 8. Đơn 2 có tau_2 = 5 <= 8 và lợi ích 8. Có hai phương án: (1) Nhánh không lấy đơn 2: Lợi ích lấy từ hàng trước: F(1, 8) = 6. (2) Nhánh có lấy đơn 2: Nhận lợi ích 8, ngân sách còn lại 8 - 5 = 3, kết hợp bài toán con tối ưu F(1, 3) = 6 (chọn đơn 1 tốn chi phí 3): tổng lợi ích là 8 + F(1, 3) = 8 + 6 = 14. Giá trị ô là max(6, 14) = 14.

- Giải thích một bước truy vết: Bắt đầu từ ô (i=3, b=8) tại table[4, 9] có giá trị 14. So sánh với hàng trên table[3, 9] (F(2, 8)) cũng có giá trị 14. Vì giá trị bằng nhau, theo quy tắc ưu tiên không lấy khi hòa, đơn 3 không được chọn; giữ nguyên b = 8 và chuyển lên xét i = 2. Tại ô (i=2, b=8), giá trị là 14 trong khi hàng trên table[2, 9] (F(1, 8)) có giá trị 6. Vì 14 > 6, đơn 2 được chọn! Ghi nhận ID 2, trừ ngân sách b = 8 - 5 = 3, lùi lên xét (i=1, b=3).

- Đối chiếu với vét cạn: 3 đơn hàng tạo ra 2^3 = 8 tập con: rỗng (tau=0, V=0); {1} (tau=3, V=6); {2} (tau=5, V=8); {3} (tau=7, V=10); {1, 2} (tau=3+5=8 <= 8, V=6+8=14 - Hợp lệ & Tối ưu); {1, 3} (tau=10 > 8 - Loại); {2, 3} (tau=12 > 8 - Loại); {1, 2, 3} (tau=15 > 8 - Loại). Kết luận: Phương án {1, 2} đạt lợi ích lớn nhất 14 trong ngân sách 8. Kết quả DP hoàn toàn trùng khớp với vét cạn (oracle).

5. Giao và hoàn tác

Trạng thái	Queue	Stack	Đã dùng	Đã nhận
Sau lập kế hoạch	[1, 2]	[]	0	0
Sau giao một đơn	[2]	[1]	3	6
Sau hoàn tác	[1, 2]	[]	0	0

6. Đầu ra và nhận xét

Đường dẫn CSV kết quả: data/MINI001/ket_qua.csv (và data/N24DCDT089/ket_qua.csv) Link notebook: TH_2026_10_09_Pluto.jl (https://github.com/phamngoctho/CTDL_GT_Julia_2026_10_09/blob/main/TH_2026_10_09_Pluto.jl)

Lỗi đã phát hiện và cách sửa: (1) Lỗi chỉ số 1-based trong bảng DP của Julia: Do chỉ số mảng trong Julia bắt đầu từ 1, nếu lập bảng kích thước (n, budget) và truy cập table[i, b] thì khi i=0 hoặc b=0 sẽ gặp lỗi BoundsError. Cách sửa: khai báo ma trận kích thước (n+1, budget+1), ánh xạ trạng thái F(i, b) tương ứng với ô table[i+1, b+1], khởi tạo hàng 1 và cột 1 bằng 0 làm điều kiện cơ sở. (2) Lỗi đưa lặp đỉnh vào BFS: Ban đầu đánh dấu đỉnh đã thăm khi dequeue, dẫn đến việc các đỉnh có nhiều cạnh kề bị enqueue nhiều lần gây tràn hàng đợi CircularQueue. Cách sửa: đánh dấu distance[v] = distance[u] + 1 và parent[v] = u ngay khi đưa đỉnh v vào hàng đợi (enqueue). (3) Lỗi mất thứ tự ưu tiên khi hoàn tác: Khi hoàn tác nếu dùng enqueue! thông thường thì đơn hàng sẽ bị đưa về cuối hàng đợi, làm đảo lộn thứ tự ưu tiên giao hàng ban đầu. Cách sửa: sử dụng enqueue_front! để đưa đơn hoàn tác trở lại đúng vị trí đầu hàng đợi FIFO.

Một chức năng thay đổi ảnh hưởng đến chức năng sau như thế nào? Toàn bộ chương trình được thiết kế theo một đường ống dữ liệu (pipeline) liên hoàn khép kín: Nếu bản đồ trong ban_do.csv thay đổi (ví dụ trong bài kiểm thử test_app khi loại bỏ cạnh (2,4) và (3,4)), cấu trúc đồ thị danh sách kề adjacency thay đổi -> hàm bfs_routes sẽ tìm ra đường đi ngắn nhất mới và số cạnh h mới -> chi phí thời gian tau = 2h + s của từng đơn hàng trong từ điển costs sẽ thay đổi (chi phí đơn 2 tăng từ 5 lên 9) -> đầu vào chi phí của thuật toán quy hoạch động knapsack thay đổi, dẫn đến bảng table thay đổi và tập đơn tối ưu được chọn đổi sang đơn 3 (thay vì đơn 1 và 2) -> danh sách các đơn đưa vào hàng đợi CircularQueue thay đổi -> trình tự giao hàng và dữ liệu xuất ra tệp ket_qua.csv thay đổi tương ứng.

Câu hỏi cuối buổi

Q1. Vì sao không được dùng dãy đã sắp giảm theo lợi ích/τ để tìm kiếm nhị phân theo ID? Nêu tiền điều kiện của hàm tìm kiếm.

Tiền điều kiện bắt buộc của thuật toán tìm kiếm nhị phân là dãy dữ liệu đầu vào phải được sắp xếp tăng dần (hoặc giảm dần) theo đúng khóa tìm kiếm (trong bài là trường Order.id tăng dần).

Nếu sử dụng dãy đã sắp giảm theo tỷ số lợi ích/tau, các bản ghi được sắp xếp theo độ ưu tiên giao hàng chứ không theo id. Khi đó, các giá trị id trong mảng nằm rải rác lộn xộn, không còn tính chất đơn điệu. Khi so sánh khóa cần tìm với phần tử giữa a[mid].id, ta không thể xác định được phần tử cần tìm nằm ở nửa bên trái hay nửa bên phải để loại bỏ một nửa không gian tìm kiếm. Việc cố tình áp dụng tìm kiếm nhị phân trên dãy này sẽ loại trừ sai nửa chứa phần tử, dẫn đến việc bỏ sót phần tử hoặc kết luận sai rằng phần tử không tồn tại (trả về nothing).

Q2. Với đơn ID 2 tới điểm 4, phục vụ 1 và bản đồ mẫu, giải thích từng thành phần của thời gian thực hiện τ. BFS có tiếp tục cho thời gian ngắn nhất nếu các cạnh mang thời gian khác nhau không?

- Công thức tính thời gian thực hiện một đơn: tau = 2h + s. Với đơn ID 2 giao tới điểm 4:

+ h = 2: Số cạnh ít nhất trên đường đi từ kho 1 tới điểm giao 4 (đường ngắn nhất là 1 -> 2 -> 4 hoặc 1 -> 3 -> 4, gồm 2 cạnh). Do mỗi cạnh tốn 1 đơn vị thời gian, lượt đi từ 1 tới 4 mất 2 đơn vị thời gian.

+ Hệ số 2: Mỗi đơn hàng là một chuyến đi độc lập: xe xuất phát từ kho 1 tới điểm giao rồi bắt buộc phải quay trở về lại kho 1. Do đó thời gian di chuyển cả đi lẫn về là 2 * h = 2 * 2 = 4 đơn vị thời gian.

+ s = 1: Thời gian dừng để phục vụ giao hàng cho khách tại điểm đến. Tổng thời gian thực hiện là tau = 2 * 2 + 1 = 5.

- Nếu các cạnh mang thời gian khác nhau (đồ thị có trọng số cạnh dương), thuật toán BFS thông thường không còn đảm bảo tìm được đường đi có tổng thời gian ngắn nhất nữa. Lý do là BFS chỉ mở rộng theo số lượng cạnh ít nhất (đồ thị không trọng số); một đường đi qua ít cạnh nhưng các cạnh đó có trọng số rất lớn thì tổng thời gian vẫn có thể lớn hơn một đường đi qua nhiều cạnh nhưng các cạnh có trọng số rất nhỏ. Để tìm đường ngắn nhất khi cạnh có trọng số khác nhau, ta phải dùng thuật toán Dijkstra.

Q3. Giải thích hai phương án tạo ô F(2,8) và vị trí của ô đó trong table Julia. Vì sao nhánh lấy đơn phải dùng hàng trước?

- Hai phương án tạo ô F(2,8): Đây là bài toán con xét tập 2 đơn đầu tiên {1, 2} với ngân sách b = 8. Đơn 2 có chi phí tau_2 = 5 <= 8 và lợi ích 8:

+ Phương án 1 (Không chọn đơn 2): Ta không nhận lợi ích của đơn 2 và không tốn chi phí của đơn 2. Lợi ích tối ưu lấy từ hàng trước tại cùng ngân sách 8, tức F(1, 8) = 6 (chỉ chọn đơn 1).

+ Phương án 2 (Có chọn đơn 2): Ta nhận lợi ích 8 của đơn 2, ngân sách còn lại là 8 - 5 = 3. Giá trị tối ưu từ bài toán con trước đó với ngân sách 3 là F(1, 3) = 6 (chọn đơn 1 tốn chi phí 3). Tổng lợi ích phương án này là 8 + F(1, 3) = 8 + 6 = 14.

Giá trị của ô là giá trị lớn nhất giữa hai phương án: F(2,8) = max(6, 14) = 14.

- Vị trí trong bảng table của Julia: Trong Julia, mảng có chỉ số bắt đầu từ 1. Hàng 1 và cột 1 được dành cho trạng thái cơ sở (i=0, b=0). Do đó trạng thái F(i, b) được lưu tại vị trí table[i+1, b+1]. Với i=2, b=8, ô F(2,8) nằm tại vị trí table[3, 9].

- Nhánh lấy đơn phải dùng hàng trước (i-1, tức table[i, ...]) vì đây là bài toán ba lô 0/1: mỗi đơn hàng chỉ được phép chọn tối đa một lần. Nếu dùng hàng hiện tại i (table[i+1, ...]), bài toán con có thể đã chứa chính đơn hàng i, dẫn đến việc một đơn hàng bị chọn lặp lại nhiều lần (chuyển thành bài toán ba lô không giới hạn - Unbounded Knapsack).

Q4. Sau khi giao ID 1 rồi hoàn tác ngay, FIFO, ngăn xếp, tổng thời gian và tổng lợi ích phải thay đổi thế nào? Nêu một bất biến kiểm tra việc không mất hoặc lặp đơn.

- Sự thay đổi trạng thái khi giao ID 1 và hoàn tác ngay sau đó:

+ Khi thực hiện giao ID 1 (deliver!): Lấy ID 1 ra khỏi đầu hàng đợi FIFO (dequeue!), danh sách chờ chuyển từ [1, 2] thành [2]. Đẩy ID 1 vào đỉnh ngăn xếp Stack (push!), lịch sử chuyển từ [] thành [1]. Tổng thời gian đã dùng tăng thêm tau_1 = 3 (từ 0 lên 3). Tổng lợi ích đã nhận tăng thêm lợi ích của đơn 1 là 6 (từ 0 lên 6). Trạng thái: Queue: [2], Stack: [1], Đã dùng: 3, Đã nhận: 6.

+ Khi thực hiện hoàn tác ngay sau đó (undo!): Lấy ID 1 ra khỏi đỉnh ngăn xếp (pop!), lịch sử trở về []. Đưa ID 1 trở lại ĐẦU hàng đợi FIFO bằng hàm enqueue_front!, danh sách chờ được khôi phục thành [1, 2] (bảo toàn đúng thứ tự ưu tiên ban đầu). Trừ thời gian đã dùng đi tau_1 = 3 (trở về 0). Trừ lợi ích đã nhận đi 6 (trở về 0). Khôi phục hoàn toàn trạng thái trước khi giao: Queue: [1, 2], Stack: [], Đã dùng: 0, Đã nhận: 0.

- Bất biến kiểm tra việc không mất hoặc lặp đơn:

Tại mọi thời điểm của phiên làm việc, tập hợp các đơn đang chờ trong hàng đợi (pending) và tập hợp các đơn đã giao trong ngăn xếp (completed) phải thỏa mãn:

1. Rời nhau hoàn toàn: Set(pending) ∩ Set(completed) == ∅ (không có đơn nào vừa đang chờ vừa đã giao).

2. Hợp của hai tập hợp luôn bằng đúng tập đơn đã chọn theo kế hoạch tối ưu: Set(pending) ∪ Set(completed) == Set(selected_ids) và tổng số phần tử luôn bảo

toàn: length(pending) + length(completed) == length(selected_ids) (đảm bảo không làm phát sinh thêm đơn lạ và không làm thất thoát bất kỳ đơn nào).

Q5. Cho biết độ phức tạp theo n,V,E,B của sắp xếp trộn, tìm kiếm nhị phân, BFS cốt lõi và DP bảng đầy đủ. Vì sao không thể khẳng định mọi BST đều tra cứu trong O(log n)?

- Độ phức tạp theo các tham số n, V, E, B:

1. Sắp xếp trộn (Merge Sort): Thời gian O(n log n) trong mọi trường hợp (tốt nhất, trung bình, xấu nhất). Bộ nhớ phụ O(n) cho mảng đệm trộn và O(log n) độ sâu ngăn xếp đệ quy.

2. Tìm kiếm nhị phân (Binary Search): Thời gian O(log n). Bộ nhớ phụ O(1) với thuật toán lặp.

3. BFS cốt lõi (trên danh sách kề): Thời gian O(V + E) (mỗi đỉnh vào/ra hàng đợi 1 lần, mỗi cạnh duyệt qua 2 lần). Bộ nhớ phụ O(V) để lưu mảng distance, parent và hàng đợi CircularQueue (nếu tính cả bộ nhớ lưu toàn bộ tuyến đường routes cho mọi đỉnh thì bộ nhớ là O(V^2)).

4. Quy hoạch động ba lô bảng đầy đủ (Knapsack DP): Thời gian O(nB) để điền bảng kích thước (n+1) x (B+1) và O(n) để truy vết tìm tập nghiệm. Bộ nhớ phụ O(nB) cho toàn bộ bảng. Đây là độ phức tạp đa thức giả (pseudo-polynomial) vì phụ thuộc vào giá trị số học của ngân sách B.

- Vì sao không thể khẳng định mọi BST đều tra cứu trong O(log n):

Chi phí tra cứu trên cây tìm kiếm nhị phân (BST) phụ thuộc trực tiếp vào chiều cao của cây h, tức là có độ phức tạp O(h). Chi phí này chỉ đạt O(log n) khi cây tương đối cân bằng (như cây AVL, Đỏ-Đen hoặc cây dựng từ trung điểm mảng đã sắp xếp). Trong trường hợp tổng quát của cây BST thông thường, nếu dữ liệu đầu vào được chèn theo thứ tự đã sắp xếp (ví dụ chèn dãy ID tăng dần 1, 2, 3, ..., n), mỗi nút mới luôn được chèn vào nhánh con bên phải, khiến cây bị suy biến hoàn toàn thành một đường thẳng (danh sách liên kết đơn). Khi đó chiều cao của cây là h = n, và chi phí tra cứu trong trường hợp xấu nhất sẽ là O(n) chứ không phải O(log n).